Chapter 19 Designing for Visibility

Omitted!

Chapter 20
Mapping Designs to Code

Code in my head ⇔ Code I wrote

Code in my head

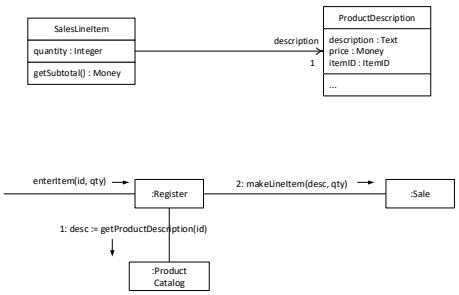
Code I wrote

Programming and Iterative, Evolutionary Development 1

- OOAD
  - Use cases + OOA/D + OOP provide an end-to-end roadmap from requirements through to code
  - Design modeling should not be taken to imply that there is no prototyping or design-while-programming
  - Gives a starting point for experimentation and discussion
  - OO design modeling provides a great base that scales up with elegance and robustness to meet the new problems
- Programming
  - Programming work is not a trivial code generation step
  - Design modeling is NOT an incomplete first step
  - During programming and testing, myriad changes will be made and detailed problems will be uncovered and resolved

Programming and Iterative, Evolutionary Development 2

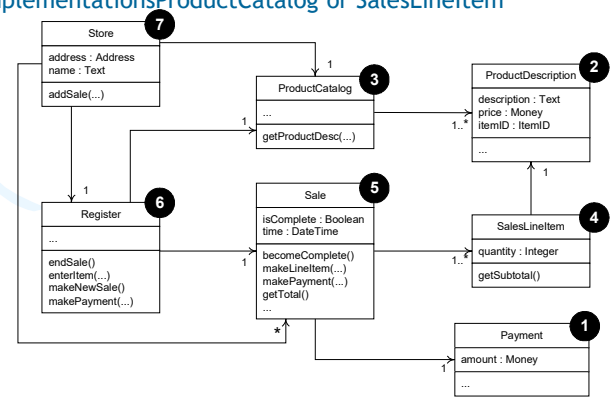
- Mapping Designs to Code (writing source code for)
  - class and interface definitions (DCD)
  - method definitions (interaction Diagrams)



Code mapping is ignored!

Order of Implementation

- Order of Implementation
  - Classes need to be implemented (and ideally, fully unit tested) from least-coupled to most-coupled
  - For example, possible first classes to implement are either Payment or ProductDescription; next are classes only dependent on the prior implementationsProductCatalog or SalesLineItem

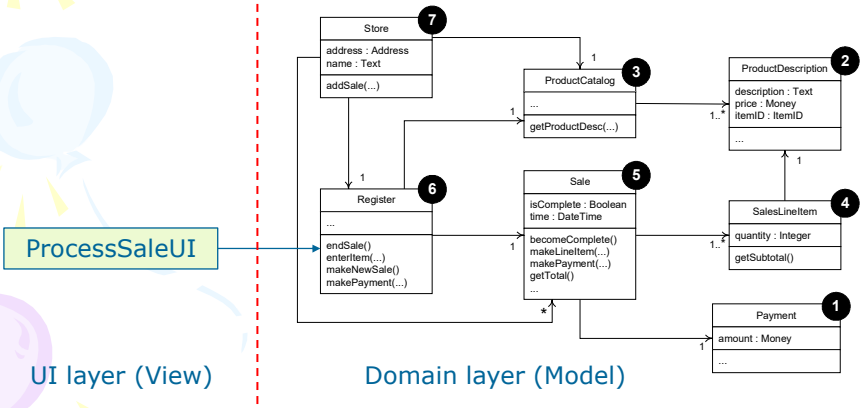


Summary of Mapping Designs to Code

- There is a translation process
  - from UML class diagrams to class definitions
  - from interaction diagrams to method bodies.
- There is still lots of room for creativity, evolution, and exploration during programming work
- Sample code mapping for this NextGen POS iteration can be found in page 377-380
- TDD (next chapter)

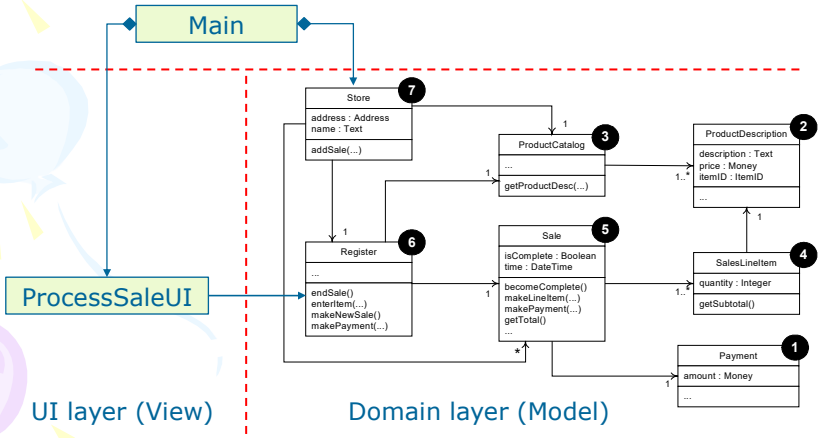
What about UI?

- Question: Where are the UI classes?



What about main?

- Question: Is there a main() program?



Chapter 21 Test-Driven Development and Refactoring

Question: Why is that TDD and Refactoring are discussed in the same chapter?

Cohesion?

12

Regression

- Question: What is regression (in CS)?

13

Regression

- Question: How can you avoid regression (errors)?

Regression Testing

14

Unit Testing

- Question: After writing a piece of code, do you need to test (manually) whether the code works? (next page)

Unit Testing

- 倚天屠龍記
  - 張無忌對乾坤大挪移第7層有19句沒練成？

Unit Testing

- Question: What do you get when unit testing is automated?

17

Introduction

- Test-Driven Development (TDD)
  - Extreme Programming (XP) promoted an important testing practice: writing the tests first.
  - It also promoted continuously refactoring code to improve its quality - less duplication, increased clarity, and so forth
  - It is also known as test-first development.
  - TDD covers more than just unit testing (testing individual components)
- Key Point: The test is written first, imagining the code to be tested is written.

18

Test-Driven Development 1

- Advantages include
  - The unit tests actually get written
  - Programmer satisfaction leading to more consistent test writing
    - The *test-last development*, also known as *Just-this-one-time-I'll-skip-writing-the-test* development, doesn't make developers feel satisfying
  - Clarification of detailed interface and behavior
    - That reflection improves or clarifies the detailed design.
  - Provable, repeatable, automated verification
    - The early painful investment pays off as the size of the AP grows
  - The confidence to change things
    - When a developer needs to change existing code written by themselves, there is a unit test suite that can be run , providing immediate feedback if the change caused an error.

19

Test-Driven Development 2

- The popular unit testing framework is the xUnit family
  - Java: JUnit; .NET: NUnit
  - JUnit is integrated into most of the Java IDEs, such as Eclipse.
- Each testing method follows this pattern (example in the **next page**)
  - Create the fixture (an object to be tested).
  - Do something to it (some operation that you want to test).
  - Evaluate that the results are as expected.

20

Test-Driven Development 2

- Question: suppose we are testing the class Foo with TDD, which of the following orders is correct?

```
class Foo {
  public void m1() {…};
  public void m2() {…};
  …
}

class testFoo {
  public void testM1() {…};
  public void testM2() {…};
}
```

(A) Write in the following order
testFoo.testM1(), testFoo.testM2(),
Foo.m1(), Foo.m2()

(B) Write in the following order
testFoo.testM1(), Foo.m1(),
testFoo.testM2(), Foo.m2()

We write only one test method, implement the solution in class under test to make it pass, and then repeat

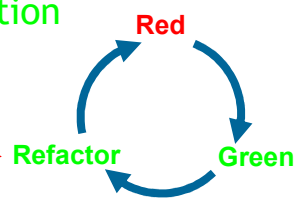
21

Test-Driven Development 3

- TDD Rhythm
  - Quickly add a test
  - Run all tests and see the new one fail
  - Make a little change
  - Run all tests and see them all pass
  - Refactor to remove duplication

Do it as quickly as possible

Takes time, but keeps green



22

Test-Driven Development 4

- Create a separate testing method for each Sale method.
  - write unit testing methods for each public method of the Sale class.
- To test method doFoo, it is an idiom to name the testing method testDoFoo.

```
public class SaleTest extends TestCase {
    // …
    // test the Sale.makeLineItem method
    public void testMakeLineItem() {
        // STEP 1: CREATE THE FIXTURE
        // -this is the object to test
        // -it is an idiom to name it 'fixture'
        // -it is often defined as an instance field rather than
        // a local variable Sale fixture = new Sale();
        // set up supporting objects for the test
        Money total = new Money( 7.5 );
        Money price = new Money( 2.5 );
```


Test-Driven Development 4

```
        ItemID id = new ItemID( 1 );
        ProductDescription desc = new ProductDescription( id, price, "product 1" );
        // STEP 2: EXECUTE THE METHOD TO TEST
        // NOTE: We write this code **imagining** there
        // is a makeLineItem method. This act of imagination
        // as we write the test tends to improve or clarify
        // our understanding of the detailed interface to
        // to the object. Thus TDD has the side-benefit of
        // clarifying the detailed object design.
        // test makeLineItem
        sale.makeLineItem( desc, 1 );
        sale.makeLineItem( desc, 2 );
        // STEP 3: EVALUATE THE RESULTS
        // there could be many assertTrue statements
        // for a complex evaluation
        // verify the total is 7.5 (2.5 + 2.5*2)
        assertTrue( sale.getTotal().equals( total ));
    }
}
```

Test-Driven Development 5

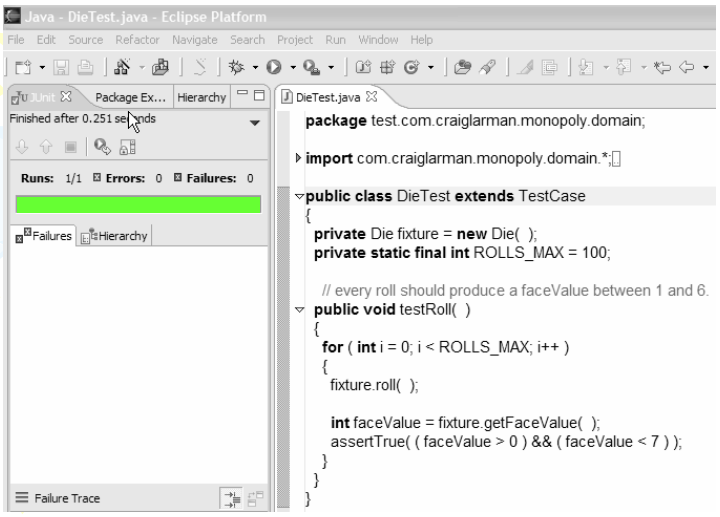
- Important: always automate testing
  - If you need to know whether a particular class is correctly written, write a unit test
  - If you need to see something, write a unit test
  - If you need to debug a particular class, write a unit test.

```
class Foo {
…
};

void main()
{
    Foo f(…);
    f.SetXXX(…);
    f.DoXXX(…);
    cout << f.GetXXX(…);
}
```

```
class Foo {
…
};

void TestFooXXX()
{
    Foo f(…);
    f.SetXXX(…);
    f.DoXXX(…);
    AssertEqual(f.GetXXX(), …);
}
```

25

IDE Support for TDD and xUnit



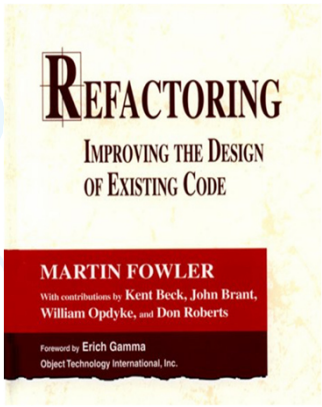
26

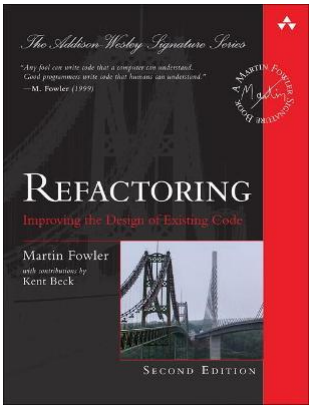
Unit testing versus Tester

- Question: You have completed unit testing for your product, do you still need a "tester" to test the product?

27

Refactoring





28

Refactoring

- Question: which of the followings is NOT a code smell (not in the list of 22 smells in the refactoring book, 2nd version)?

(A) Comments
(B) Data class
(C) Duplicated code
~~(D) Brain surgery~~ ⇨ Shotgun Surgery
(E) Feature Envy
(F) Message Chains
(G) Repeated Switch

What do these smells mean?

Refactoring 1

- Motivation (next 3 pages)

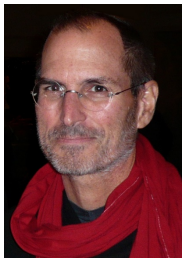
As code quickly grows

A real example in the next page

31

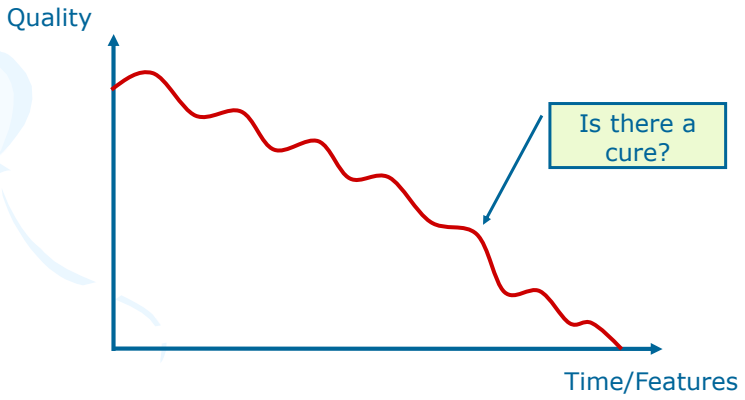
As code quickly grows

- Steve Jobs (賈柏斯)
  - Founded Apple in 1976.
  - Worth 100 million dollars when he was 25 (1980) .
  - Forced to leave Apple in 1985.
  - Founded Next Computer in 1985.
  - Founded Pixar in 1986.
  - Returned to Apple in 1997.
  - Introduced iMac in 1998.
  - Introduced iPhone in 2007.
  - Died in 2011.



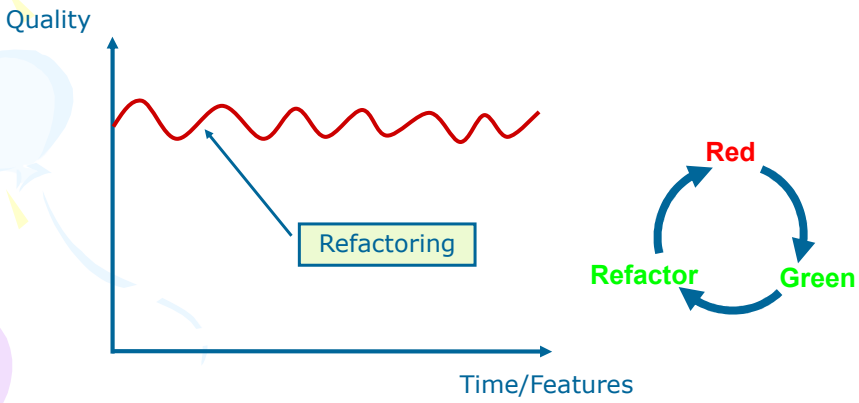
32

A Typical Code Development Scenario



33

A Typical Code Development Scenario



34

Refactoring

- Refactoring
  - Is a structured, disciplined method to rewrite or restructure existing code without changing its external behavior.

35

Refactoring

- Question: What is a "reasonable" sequence of refactoring the code on the next page into the code on next, next page?

  (1) Do X and Y at the same time

  (2) Doesn't matter

  (3) Do X and then Y

  (4) Do Y and then X

Refactoring

```
class Foo {
public:
    void SetValue(int);
    int GetValue();
private:
    int value;
};
```

```
// Some other class
void doSomething() {
  Foo *foo = new Foo();
  …
  int  v = foo->GetValue();
  if (v >= 100)
      foo->SetValue(v * 0.8);
  else
      foo->SetValue(v * 0.9);
  …
}
```

37

Refactoring

```
class Foo {
public:
    void SetValue(int);
    int GetValue();
    void Discount()
    {
        if (value > 100)
            value *= 0.8;
        else
            value *= 0.9
    }
private:
    int value;
};
```

```
// Some other class
void doSomething() {
  Foo *foo = new Foo();
  …
  foo->Discount();
  …
}
```

Step X: add Discount() method()

Step Y: replace the original code with a method call

Case 1: refactor production code only X → Y
Case 2: refactor production code & test code (next)
Case 3: TDD (next)

Refactoring

```
class Foo {
public:
    void SetValue(int);
    int GetValue();
    void Discount()
    {
        if (value > 100)
            value *= 0.8;
        else
            value *= 0.9
    }
private:
    int value;
};
```

Step X: add Discount() method()

```
// Some other class
void doSomething() {
  Foo *foo = new Foo();
  ...
  foo->Discount();
  ...
}
```

Step Y: replace the original code with a method call

Case 2: refactor production code & test code

1. Step X.
2. Add testDiscount() to FooTest (possibly copying some test code from testDoSomething()
3. Step Y.
4. Refactor testDoSomething() to exclude test code related to Discount()

Refactoring

```
class Foo {
public:
    void SetValue(int);
    int GetValue();
    void Discount()
    {
        if (value > 100)
            value *= 0.8;
        else
            value *= 0.9
    }
private:
    int value;
};
```

Step X: add Discount() method()

```
// Some other class
void doSomething() {
  Foo *foo = new Foo();
  ...
  foo->Discount();
  ...
}
```

Step Y: replace the original code with a method call

Case 3: TDD (refactor write test code first)

1. Add testDiscount() to FooTest (possibly copying some test code from testDoSomething()
2. Step X.
3. Refactor testDoSomething() to exclude test code related to Discount()
4. Step Y.

Refactoring 1

- The essence of refactoring
  - Applying small behavior preserving transformations (each called a 'refactoring'), one at a time.
  - After each transformation, the unit tests are re-executed to prove that the refactoring did not cause a regression (failure).
  - Each refactoring is small, but a series of transformations — each followed by executing the unit tests again — can, of course, produce a major restructuring of the code and design, all the while ensuring the behavior remains the same

Refactoring 2

- The activities and goals of refactoring
  - Remove duplicate code
  - Improve clarity
  - Make long methods shorter
  - Remove the use of hard-coded literal constants
- Code that's been well-refactored is short, tight, clear, and without duplication — it looks like the work of a master programmer.
- Code that doesn't have these qualities smells bad or has code smells
  - Code smells is a metaphor in refactoring — they are hints that something may be wrong in the code
  - Code stench — truly putrid code that requires for clean up

Refactoring 3

- Some code smells include
  - Duplicated code
  - Big method
  - Class with many instance variables
  - Class with lots of code
  - Strikingly similar subclasses
  - Little or no use of interfaces in the design
  - High coupling between many objects
- The remedy to smelly code are the refactorings.
- Like patterns, refactorings have names, such as Extract Method.
  - There are about 100 named refactorings

43

Refactoring 4

- Examples of Refactorings

Refactoring	Description
Extract Method	Transform a long method into a shorter one by factoring out a portion into a private helper method.
Extract Constant	Replace a literal constant with a constant variable.
Introduce Explaining Variable (specialization of Extract Local Variable)	Put the result of the expression, or parts of the expression, in a temporary variable with a name that explains the purpose.
Replace Constructor Call with Factory Method	In Java, for example, replace using the new operator and constructor call with invoking a helper method that creates the object (hiding the details).

44

Refactoring 5

- Example of the Extract Method refactoring
  - We can make the takeTurn method shorter, clearer, and better supporting High Cohesion by extracting that code into a private helper method called rollDice

```
public class Player
{
    private Piece  piece;
    private Board  board;
    private Die[]  dice;
    // …

    public void takeTurn()
    {
        // roll dice
        int rollTotal = 0;
        for (int i = 0; i < dice.length; i++)
        {
                dice[i].roll();
                rollTotal += dice[i].getFaceValue();
        }
        // move to the target square
        Square newLoc = board.getSquare(piece.getLocation(), rollTotal);
        piece.setLocation(newLoc);

    }
} // end of class
```

45

Refactoring 6

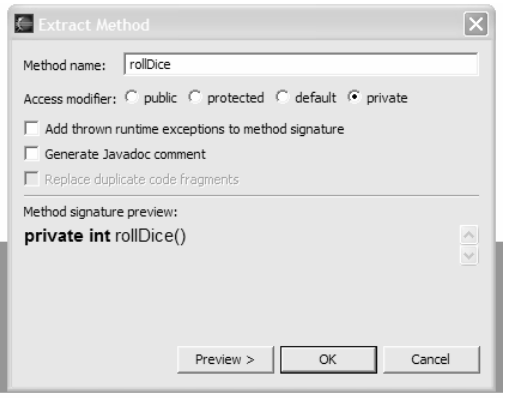
- The code after applying the Extract Method refactoring

```
public class Player
{
    private Piece  piece;
    private Board  board;
    private Die[]  dice;
    // …

    public void takeTurn()
    {
        // the refactored helper method
        int rollTotal = rollDice();

        Square newLoc = board.getSquare(piece.getLocation(), rollTotal);
        piece.setLocation(newLoc);
    }

    private int rollDice()
    {
        int rollTotal = 0;
        for (int i = 0; i < dice.length; i++)
        {
            dice[i].roll();
            rollTotal += dice[i].getFaceValue();
        }
        return rollTotal;
    }
} // end of class
```

46

IDE Support for Refactoring: Eclipse IDE 1

```
public Player(String name, Die[] dice, Board board)
{
  this.name = name;
  this.dice = dice;
  this.board = board;
  piece = new Piece(board.getStartSquare());
}

public void takeTurn()
{
  // roll dice
  int rollTotal = 0;
  for (int i = 0; i < dice.length; i++)
  {
    dice[i].roll();
    rollTotal += dice[i].getFaceValue();
  }

  Square newLoc = board.getSquare(piece.getLocation(), rollTotal);
  piece.setLocation(newLoc);
}
```



Visual studio 2005/2008 also supports refactoring

IDE Support for Refactoring: Eclipse IDE 2

```
public void takeTurn()
{
  int rollTotal = rollDice();

  Square newLoc = board.getSquare(piece.getLocation(), rollTotal);
  piece.setLocation(newLoc);
}

private int rollDice()
{
  // roll dice
  int rollTotal = 0;
  for (int i = 0; i < dice.length; i++)
  {
    dice[i].roll();
    rollTotal += dice[i].getFaceValue();
  }
  return rollTotal;
}
```

48

Refactoring 7

- The Introduce Explaining Variable refactoring
  - It clarifies, simplifies, and reduces the need for comments
- Before introducing an explaining variable.

```
// good method name, but the logic of the body is not clear
boolean isLeapYear( int year ) {
    return( ( ( year % 400 ) == 0 ) || ( ( ( year % 4 ) == 0 )
        && ( ( year % 100 ) != 0 ) ) );
}
```

- After introducing an explaining variable.

```
// that's better!
boolean isLeapYear( int year ) {
    boolean isFourthYear = ( ( year % 4 ) == 0 );
    boolean isHundrethYear = ( ( year % 100 ) == 0);
    boolean is4HundrethYear = ( ( year % 400 ) == 0);
    return(is4HundrethYear||(isFourthYear && !isHundrethYear));
}
```

49